C++ contracts implementation strategies

Document number	P3267R1
Date	2024-05-22
Reply-to	Peter Bindels <dascandy@gmail.com>
Co-authors	Tom Honermann <tom@honermann.net>
Targeted subgroups	SG21, SG15

§ 1 Introduction

This document hopes to capture existing thoughts on implementation strategies for Contracts in C++ (P2900), so that we can refer to them in one place and hopefully expand the set if somebody comes up with a genuinely different approach.

This paper is extracted in part from D3250R0.

A related paper is P3264, which looks at implementation strategies from a theoretical point of view with regards to what it implies for evaluation counts.

§ 1.1 Revision history

R1:

- Add approach 5: delayed checking
- Add approach 6: run-time selection
- Add approach 7: load-time selection
- Add entry on evaluation count guarantees
- Minor restructuring

§ 1.2 Example code for discussion

For the examples below, we will consider a function `f()` that looks as follows:

```
int f(int arg)
  pre(arg > 5)
  pre(arg < 2000)
  post(rv: rv % 2 == 0)
{
  return arg * 2;
}
```

and a function `g()` invoking it that looks as follows:

```
int g() {
```

```
  return f(25);
}
```

plus a pointer indirection used by `h()`:

```
using function_pointer = int(*)(int);
function_pointer fp = &f;
int h() {
  return fp(40);
}
```

§ 1.3 Pseudocode for pseudofunctions `check()` and `imply()`

```
consteval bool contract_should_check();
consteval bool contract_should_imply();

#define check(check_content) \
    if (not contract_should_check() or check_content()) {} else \
       if (quick_enforce) std::terminate() else
         contract_violation(check_content)

#define imply(f) if (not contract_should_imply()) {} else __builtin_assume(f)
```

The intent of having these two sides separate is so that we can illustrate where we would be allowing use of which property of the logical `assert`. An assert normally does both of these in a single go.

§ 1.4 Note on whether checks occur and what they imply

P2900 allows space for different contract checking facilities. It provides the space in 3.5.2 to select either "Ignore", "Observe" or "Enforce" semantics. These boil down to at the place where checking would be done, whether to check it at all, and in case a check is indeed done whether to consider a failure of the check a program-ending event.

Name	Check	Imply
Ignore	No	No
Observe	Yes	No
Enforce	Yes	Yes
Quick_Enforce	Yes	Yes
Assume	No	Yes

With regards to this paper, we consider all of these to be equivalent. For Ignore and Observe, the "implication" part of contract results is empty, and for Ignore the "checking" part of contracts is empty, too. The only thing this paper hopes to clarify and provide nomenclature for is the locations and structures around contracts structures, not what these locations exactly contain.

In the approaches below, we assume that the semantics for contract checking match between different translation units, making it irrelevant whether the functions are defined in one or multiple TUs. See below for a discussion on that.

§ 2 The approaches

§ 2.1 Fully callee-side checks

The most simple way to add contracts to C++ implementations, and one that is most similar to what we have seen before, is to convert a function with preconditions and postconditions into a function that does all its contracts work inside the function implementation:

```
int f(int arg)
{
  check(arg > 5);
  imply(arg > 5);
  check(arg < 2000);
  imply(arg < 2000);
  int rv = arg * 2;
  check(rv % 2 == 0);
  imply(rv % 2 == 0);
  return rv;
}

int g() {
  return f(25);
}

int h() {
  return fp(40);
}
```

As a simple implementation, it works fine. It checks all contracts, but derives little value in the callers to be optimized on. Likewise, in `f()`, the checks can only be optimized out when full visibility is available to the compiler, such as for local functions, or at link time. If the implications survive to link-time optimization stages, they can be used there to inform the compiler about further potential optimization. This does place even more effort on the monolithic linker step.

The compiler is likely able to eliminate the post-check in `f()`, as it derives from the inputs and operations, but the imply corresponding to it does not travel to the callers.

§ 2.2 Caller-side checks, callee-side assumes, post-check as original ABI entry point.

This approach is the first that lends itself to optimization on the side of the caller, as it moves the checks to a place where implications can exist that subsume them, or where other information is available allowing them to be elided anyway.

```
int f(int arg)
{
  imply(arg > 5);
  imply(arg < 2000);
  int rv = arg * 2;
  check(rv % 2 == 0);
  return rv;
}

int g() {
  check(25 > 5);
  check(25 < 2000);
  int rv = f(25);
  imply(rv % 2 == 0);
  return rv;
}

int h() {
  return fp(40);
}
```

In this approach, there is no ABI change, and contracts are able to be used for optimization. Within a full C++ program, though, there are already some holes in the safety net of contracts, as the function pointer used by `h()` is not checking the contract at all. The function pointer does not contain contract knowledge, so `h()` cannot check it, and in this approach `f()` is kept as the post-check entry point making it not check it either.

Optimization wise, this offers much space, as the pre contract can be eliminated entirely in `g()`.

§ 2.3 Caller-side checks, callee-side implies, pre-check as original ABI entry point.

Similar to approach 2, we check on caller side, and we imply on callee side. Unlike the last though, we add a separate thunk as the original ABI entry point, and we mark the original entry point with an ABI-changed name.

```
int f(int arg)
{
  check(arg > 5);
  check(arg < 2000);
  int rv = f@post-check(arg);
  imply(rv % 2 == 0);
  return rv;
}

int f@post-check(int arg)
{
  imply(arg > 5);
```

```
  imply(arg < 2000);
  int rv = arg * 2;
  check(rv % 2 == 0);
  return rv;
}

int g() {
  check(25 > 5);
  check(25 < 2000);
  int rv = f@post-check(25);
  imply(rv % 2 == 0);
  return rv;
}

int h() {
  return fp(40);
}
```

The call in `h()` now calls to `f(int arg)` which does the contract checks on its behalf, and then indirects to `f@post-check`. Implications are available everywhere they are usable, giving the optimizer space in `f@post-check` to remove the check, and to remove the checks in `g()`. It is not able to remove the checks in `f()` though, as those can come from anywhere that did not perform checks, and as such using `h()` will necessarily run the checks once.

To note is that with contract checks disabled, the function `f()` will still output the `f@post-check` entry point, both pointing to the same entry point. Similarly, a caller with disabled contract checks will emit relocations against `f()`. The rationale for these details follows near the end.

§ 2.4 Double-sided checking

```
int f(int arg)
{
  check(arg > 5);
  imply(arg > 5);
  check(arg < 2000);
  imply(arg < 2000);
  int rv = arg * 2;
  check(rv % 2 == 0);
  imply(rv % 2 == 0);
  return rv;
}

int g() {
  check(arg > 5);
  check(arg < 2000);
  int rv = f(25);
  check(rv % 2 == 0);
  imply(rv % 2 == 0);
```

```
  return rv;
}

int h() {
  return fp(40);
}
```

Similar to approach 1, in this approach we retain the original ABI, but we do checks on both sides of a function call. At first this looks superfluous, but in practice it may end up working as well as approach 1, with the benefit that we now do get implications for return values on caller side too. The pre checks on the caller side and post checks on callee side should elide, leaving only the pre checks on callee side and post checks on caller side, while we do get full benefit of the implies.

§ 2.5 Delayed checking

```
template <typename T>
struct contract_raii_handle {
  contract_raii_handle(T&& lambda)
  : _check(std::move(lambda))
  , result(2)
  {
    register_active_contract(this);
  }
  ~contract_check_raii_handle() {
    unregister_active_contract(this);
  }
  bool operator()() {
    if (result == 2) {
      result = _check();
    }
    return (bool)result;
  }
};

auto add_pending_contract_check(auto lambda) {
  return contract_raii_handle<decltype(lambda)>(std::move(lambda));
}

int f(int arg)
{
  auto pre_handle_1 = add_pending_contract_check([=]{ return arg > 5; });
  auto pre_handle_2 = add_pending_contract_check([=]{ return arg < 2000; });
  return arg * 2;
}

int g() {
  check(arg > 5);
  check(arg < 2000);
  int rv = f(25);
```

```
  auto post_handle_1 = add_pending_contract_check([=]{ return rv % 2 == 0; });
  return rv;
}

int h() {
  return fp(40);
}
```

Unlike all approaches so far, this approach does not actually run any contract checks. The only thing it does is create pending contracts, and then destroy them again, without evaluating them. The goal of this implementation approach is to at any point have a stack of applicable contracts that can be checked, if an external user so desires. This could be done in a sampling manner by a profiler, on breakpoint by a debugger, or on direct user trigger.

The syntax above mirrors a C++-esque implementation of this idea. The exact displayed solution above would have overhead, but similar implementations can be made within (for example) DWARF debugging statements, a construct similar to exception handling frames, or attached debug info for functions, which can likely do the same thing with low to zero runtime overhead.

This solution in particular allows for very expensive contracts to be added, and validated only when in scope of an interesting event, such as a break point or a crash.

§ 2.6 Run-time selection of semantic

```
int f(int arg)
{
  bool __run_checks = should_check(__func__);
  if (__run_checks) {
    check(arg > 5);
    check(arg < 2000);
  }
  int rv = arg * 2;
  if (__run_checks) {
    check(rv % 2 == 0);
  }
  return rv;
}

int g() {
  return f(25);
}

int h() {
  return fp(40);
}
```

Many programmers rely on package managers to provide builds of packages they depend on. This is particularly prevalent in Linux ecosystems where the RPM and DPKG managers are commonly used, but is also applicable for other platforms where package managers such as

vcpkg are used. These package managers typically only provide "release" builds (with NDEBUG defined and assert therefore disabled); users that want a "debug" build (with assert enabled) are on their own to procure such a build themselves. Significant costs would be imposed on package maintainers, and significant complexity imposed on package consumers, if package managers were expected to provide distinct contracts-enabled and contracts-disabled versions of all of their packages. Contracts offer an opportunity to provide an option to select the contract semantic at program load time.

A simple approach to enable such run-time selection is to emit callee-side contract checks that are evaluated based on a query function, indicating whether this evaluation of the function should check the contracts. This offers the ability to check the contracts only some of the time, while retaining the guarantee that postconditions are only checked if preconditions are. It does invoke a global function every time the function is entered, which can be a too large overhead in some cases.

§ 2.7 Load-time selection of semantic

```
static int f__check(int arg)
{
  check(arg > 5);
  check(arg < 2000);
  int rv = arg * 2;
  check(rv % 2 == 0);
  return rv;
}

static int f__nocheck(int arg)
{
  int rv = arg * 2;
  return rv;
}

static void *f__resolver() {
  // As an example of a condition to select on
  // This will be checked on the first invocation of f().
  return getenv("mustgofaster") == nullptr ? &f__check : &f__nocheck;
}

int f(int arg) __attribute__((ifunc("f__resolver")));

int g() {
  return f(25);
}

int h() {
  return fp(40);
}
```

Switching from being able to choose whether to check contracts per evaluation, to per function, gives us the room to deliver a more performant version of contract evaluation. A

simple approach to enable such load-time selection is to emit callee-side contract checks that are evaluated based on the state of a global variable initialized by an environment variable during program startup. The overhead imposed by frequent checks of a global variable is unlikely to make this a palatable option however.

Existing linker features can be employed to decrease the overhead of such an approach. For example, GNU indirect function support (ifunc) could be employed to emit a symbol that is resolved on first use to either the checked entry point (when a checked semantic is selected) or an unchecked entry point (for the ignore semantic). Such mechanics are used today to provide support for function muliversioning. A more sophisticated dynamic linker/loader could resolve symbol references to either checked or unchecked function versions at load time as part of its relocation processing.

Use of existing linker features as described above might still be insufficient to satisfy the performance demands of package maintainers; particularly if function inlining is significantly impacted. Experimentation would be necessary to determine how much overhead would be imposed in practice.

If run-time selection overhead proves to be too expensive for "release" builds, it might still be useful for "debug", "release+assert+contracts", or "test" builds to allow selective enablement of (subsets of) contract checks.

§ 3 Separate compilation and varying compile flags

So far we have looked at the impact of varying approaches from the point of view of one or multiple TUs being compiled, with or without link-time optimization, but always resulting in a single program image and with consistent use of compilation flags. While this is a view that the C++ standard supports, it does not necessarily match reality, and depending on the platform a small minority to a large majority of users will diverge from this idealized point of view.

Approach 1: Only the implementer of a function can decide what amount of checking is done.

Approach 2: Differing compile flags would potentially cause contracts to go unchecked, but still assumed.

Approach 3: Contracts would always be checked caller side, and implied callee side, with no chance of a check being missing. When checking is enabled on both sides, it acts as described above. If it is disabled on both sides, it does not check anything (but by choice, so considered safe).

In case the caller has contract checks turned off, but the callee has them turned on, the caller will generate a relocation against the plain `f()` entry point. The callee's code generation then inserts the contract check, so that the caller does not need to consider it, but the callee does get its guarantees.

If it is enabled only at the caller side, the contracts are checked, but not implied (safe). The caller will generate a relocation against the `@post-check` entry point, which requires the callee to be compiled with a compiler supporting this ABI change, but not necessarily with contracts turned on.

Only if some of the implementations are compiled with "assume" semantics will it be possible to skip the check and still get the imply.

Approach 4 results in a situation where both sides always generate contract checks. Contract checking can be turned on and off on either side with no impact on the other side, and checks are guaranteed if they are enabled on either side.

Approach 5 will result in contracts being emitted only in the functions that were compiled with contract checks, similar to other debug information.

Approach 6 and 7 will result in contracts being possible (runtime / load time choice) only on functions that were compiled with contract checks enabled.

§ 4 Weak functions and varying compile flags

The foregoing discussion applies to all functions that reside definitely in a single TU. In case of weak functions, such as inline functions, inline defined member functions, template instantiations etc., instantiations can be emitted in multiple translation units. When all compile flags match and no other problems exist in the code base, this will lead to multiple identical symbols (usually in separate sections), one of which will be linked in to the final executable semi at random. As they are identical, this has no impact.

In case of contracts and their enforcement settings differing between translation units, this means that weak functions defined in translation units where some are compiled with one setting, and some with another, will lead to what is logically an ODR violation. As the functions don't have any difference as far as the C++ abstract machine is concerned - any valid execution of them will result in the same behavior - they can be considered as equivalent as far as ODR violations are concerned.

In all variants of approaches listed above, the result will be that one of the variants of contract execution is selected at random. Only in approach 2 is there a possibility to create a situation where a contract is implied on one side, but never checked on the other, see the previous chapter for details on how this works.

§ 5 Evaluation count guarantees

With regards to promises made by contracts, it is relevant to know how many evaluations we are guaranteed to do for the documented implementation approaches. For the first table we will assume the whole program is compiled with the same execution strategy.

Approach	Minimum number of evaluations	Maximum number of evaluations
Callee-side checks	1	1
Caller-side checks, post-check	0	1
Caller-side checks, pre-check	1	1
Double-sided checking	2	2
Delayed checking	0	1 (∞)
Run-time checking	0	1

Load-time checking	0	1

Delayed checking is triggered only when instructed externally, which is not related to the rate of function entry/exit. As such, a sampling profiler could trigger a given functions' preconditions an infinite number of times. Within the approach, the ability exists for the RAII object to cache the result of a contract evaluation to hard limit this to one. As unbounded evaluations could result in UB, it is strongly recommended to do such a thing.

Within mixed compilation modes, it is possible to reduce the minimum number of evaluations to zero for all. The maximum number does not change.

§ 6 Precondition vs postcondition check mismatches

It is possible for some approaches to allow a mismatch between preconditions and postconditions to be evaluated. Right now, P2900 does not forbid this.

In the case that we have approach 2 (Caller-side checks, callee-side assumes, post-check as original ABI entry point) we can have a mismatch between precondition check and postcondition check counts. It happens when one half is compiled with contracts and the other is compiled without, resulting in contracts being checked on entry but not exit, or inversely on exit but not on entry. Within this approach it is not fixable.

In case of approach 5 (delayed checking), it is likely that some contracts are not evaluated. In particular, when some contracts are checked externally in a function, it does not easily extend to making sure that other contracts in that same invocation are also evaluated. As such, it will usually result in postconditions not being evaluated, while preconditions sometimes are.

All other approaches guarantee balance between precondition and postcondition checks.

§ 7 Acknowledgements

Major thanks to all of SG21 for the work done on contracts so far and for the active discussions on Mattermost and the Reflector; this paper would not be possible without a very well formed P2900 to write it against, nor would it be complete or accurate without the feedback so provided.

Thanks to Corentin Jabot, Tom Honermann and others for reviewing this paper before publishing, and pointing out some confusing errors before publishing.